

ZenoDEX Whitepaper: Full-System Architecture, Public Assurance Boundary, and Proof Agenda

Dana Edwards

April 2026

Abstract

ZenoDEX is intended to be a proof-carrying exchange architecture rather than a collection of isolated trading scripts. The intended full system spans integer spot execution kernels, theorem-backed canonical winner relations, fail-closed Tau and ESSO guard layers, settlement and oracle certificate planes, bounded stablecoin and perpetual-risk slices, narrow replayable runtime adapters, and a public assurance surface that can be rebuilt from a clean checkout. This whitepaper describes that full target architecture, states the current public assurance boundary, and names the remaining proof obligations required to close the gap between the present public claim and the end-state system.

Contributions. This whitepaper contributes:

1. a decomposition of ZenoDEX into execution, guard, certificate, risk, runtime, and assurance planes;
2. an explicit evidence discipline that separates proved, contract-backed, implemented, replay-backed, and still-open claims;
3. a public-safe guarantee-and-assumption matrix for the major protocol surfaces;
4. a whole-system proof agenda that states what the architecture still needs beyond the current RC1 public surface.

1 System thesis

The intended full architecture is

$$\text{ZenoDEX} := \text{Exec} \wedge \text{Guards} \wedge \text{Cert} \wedge \text{Control} \wedge \text{Oracle} \wedge \text{zUSD} \wedge \text{Perps} \wedge \text{Runtime} \wedge \text{Assure}.$$

Standard reading: the system is the conjunction of value-moving execution, fail-closed guards, certificate-carrying settlement and oracle lanes, monetary and derivatives risk layers, narrow runtime adapters, and replayable assurance artifacts. Practical consequence: the whole system cannot be judged only by the spot math or only by the API shell. The architecture claim is inherently cross-layer.

The corresponding public discipline is

$$\text{PublicClaim} \subset \text{EvidenceFrontier} \subset \text{FullTargetArchitecture}.$$

Standard reading: the public release claim is narrower than the strongest current evidence frontier, and the evidence frontier is narrower than the full intended architecture. Practical consequence: this paper can describe the whole target system in detail without pretending that every described surface is already proved or release-backed.

2 Evidence discipline

The governing evidence order is

$$\text{proved} > \text{contract} > \text{implemented} > \text{replay_backed} > \text{hypothesis}.$$

Standard reading: a claim may only be stated at the strongest replayable support the repository actually carries. Practical consequence: this paper treats theorem-bearing spot slices, manifest-backed contract surfaces, and replayable release evidence differently rather than collapsing them into one generic “verified” label.

The paper uses four recurring claim shapes:

$$C \neq \emptyset \rightarrow \exists! w \in C \text{ such that } \forall c \in C, \text{Key}(w) \leq \text{Key}(c),$$

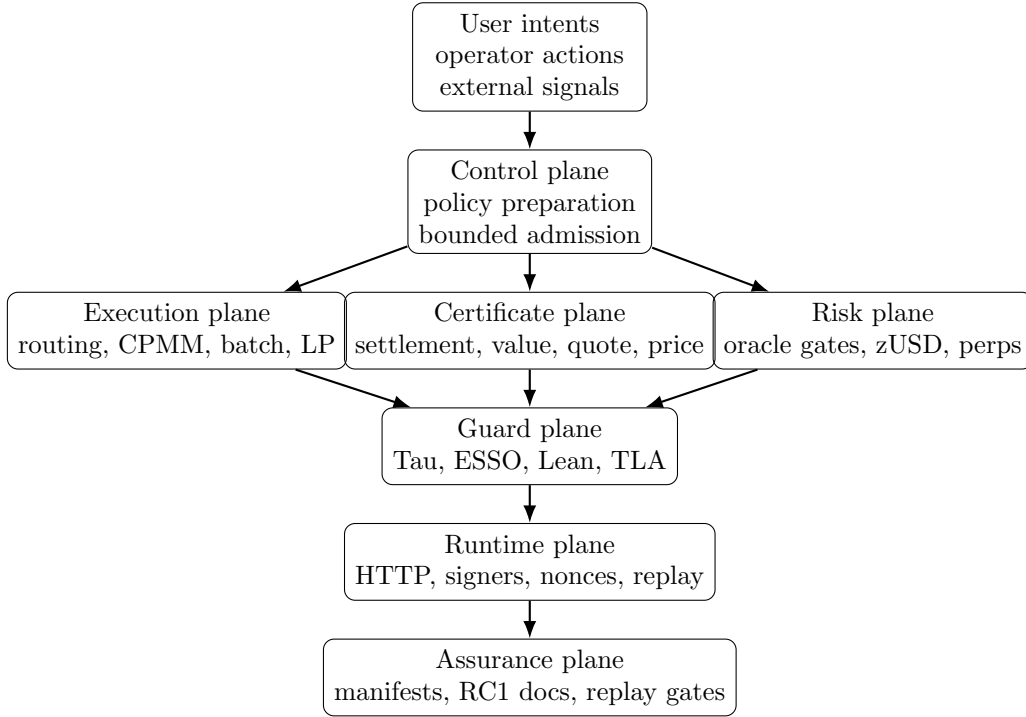


Figure 1: The intended full ZenoDEX stack. Execution authority flows only through fail-closed guards and narrow runtime adapters.

$\text{CertificateOK} \rightarrow \text{CanonicalPayload} \wedge \text{ReplayCheckable},$

$\text{RiskTransitionAllowed} \rightarrow \text{OracleAligned} \wedge \text{BoundedMove} \wedge \text{LocalInvariantOK},$

$\text{ReleaseClaimOK} := \text{CleanTree} \wedge \text{ScopeFrozen} \wedge \text{ReplayGreen} \wedge \text{ExclusionsHonest}.$

Standard reading: canonical winner, proof-carrying packets, bounded risk transitions, and replay-backed release posture are the four main logical forms used across the system. Practical consequence: the assurance story is compositional. The paper does not need one single monolithic theorem to still be technically coherent.

3 Control plane

The control plane is the whole-system layer that turns raw operator or user requests into admissible execution candidates. It is narrower than the advisory layer, but broader than the execution core. Its intended law is

$\text{AdmittedIntent} := \text{NormalFormOK} \wedge \text{SignatureOrWitnessOK} \wedge \text{NonceOK} \wedge \text{KindSupported} \wedge \text{BoundaryContractOK}.$

Standard reading: before the execution plane is even allowed to compare route or settlement candidates, ingress must already be normalized, authenticated, nonce-consistent, supported by the current release contract, and within the documented runtime boundary. Practical consequence: the system should be reviewed as *admit then optimize*, not as *optimize then try to reject dangerous payloads later*.

The public-safe control responsibilities are:

1. normalize user intents into canonical payloads;
2. bind signatures, witness packets, and nonces to those payloads;
3. reject unsupported action families or malformed envelope shapes;
4. carry forward only the bounded action language that the release surface actually supports.

This layer is therefore the bridge between the runtime plane and the execution plane. A DEX paper that omits it would miss how replay safety and canonicity are coupled in practice.

4 Execution plane

4.1 Spot routing and canonicity

The public execution core is built around explicit candidate families and total keys. The intended winner semantics is

$$\text{Winner}(C) := \arg \min_{c \in C} \text{Key}(c).$$

Standard reading: on a bounded emitted candidate family, the chosen route or batch outcome is the unique minimum under an explicit total key. Practical consequence: deterministic replay is upgraded from “same loop order” to a semantic canonicity claim.

The strongest current public posture is theorem-bearing on bounded candidate families rather than universal over all conceivable route generators. The key public references are the Lean winner theorems and the release-facing exact-in, exact-out, and batch certificate surfaces described in:

- `lean-mathlib/Proofs/BatchAuctionCanonical.lean`
- `lean-mathlib/Proofs/ZenoDEXExactInTrueKeyWinner.lean`
- `docs/RC1_VERIFIED_SURFACE_MATRIX.md`

4.2 CPMM and accounting

The spot core also relies on explicit integer arithmetic contracts:

$$\text{amount_out} < \text{reserve_out}, \quad K_{\text{after}} \geq K_{\text{before}} \text{ in the zero-fee integer reference model,}$$

with fee-side accounting shaped by

$$\text{fee_conserved} \wedge \text{dust_carry_bounded}.$$

Standard reading: reserve-bounded swap execution and exact fee accounting are first-class contracts, not informal expectations. Practical consequence: any whole-system settlement or runtime claim inherits these accounting constraints as preconditions.

4.3 Batch execution and LP semantics

Batch execution adds two additional structures:

$$\text{ValidOutcome}(B) \rightarrow \text{CanonicalBatchWinner}(B) \wedge \text{SettlementMeaningful}(B),$$

$$\text{LPRoundTrip} \rightarrow \text{NoImmediateValueCreation}.$$

Standard reading: valid batch outcomes must produce canonical winners and meaningful settlements, and LP mint/burn semantics should reject immediate value-creation folklore. Practical consequence: the paper treats batch execution and LP mechanics as part of the same execution plane, not as detached utilities.

5 Guard plane

The guard plane is intentionally redundant. ZenoDEX relies on several distinct kinds of guard rails:

1. Tau specs for fail-closed stream and policy boundaries;
2. ESSO kernels for bounded transition-system verification and solver cross-checking;
3. Lean proofs for canonicity, accounting, and algebraic helper laws;
4. bounded TLA/TLC shadow models for release-facing temporal claims.

The role of this plane is expressed by

$$\text{ExecutionAuthority} \rightarrow \text{GuardedByTauOrKernelOrProofBackedShell}.$$

Standard reading: authority-bearing transitions should only occur when they are filtered by an explicit public guard surface. Practical consequence: the architecture is not satisfied by tests alone; it requires a layered acceptance posture.

6 Certificate plane

The certificate plane is where the architecture becomes explicitly proof-carrying rather than merely deterministic.

6.1 Quote and settlement certificates

The guiding law is

$$\text{SettlementOK} \rightarrow \text{CanonicalDeltaTable} \wedge \text{QuoteOrKernelWitnessBound} \wedge \text{ReplayVerifiable}.$$

Standard reading: accepted settlement must carry canonical deltas, bind fills to quote or kernel witnesses, and remain replay-checkable. Practical consequence: settlement is a certified boundary object, not just a serialized runtime decision.

This certificate family includes:

- quote receipts and their verification surfaces;
- settlement strong certificates;
- end-to-end settlement certificate packets;
- value packets and endogenous LP value packets;
- price provenance and signed attestation packets.

6.2 Composition discipline

The composition law across packet and kernel boundaries is

$$\text{SharedFactInA} = \text{MirrorOfFactInB} \quad \text{must be preserved after every relevant transition.}$$

Standard reading: any fact shared across modules must be mirrored and kept synchronized inductively, not repaired later by convention. Practical consequence: composition is part of the proof surface, not only part of the engineering style guide.

7 Oracle and price plane

ZenoDEX treats price and oracle material as a first-class plane rather than as ambient input. The intended safety contract is

$$\text{PriceInputAccepted} \rightarrow \text{FreshnessBound} \wedge \text{SourceBound} \wedge \text{PendingStateConsistent} \wedge \text{ProvenancePacketCoherent}.$$

Standard reading: accepted price material must satisfy freshness limits, source restrictions, pending-vs-active state consistency, and provenance coherence. Practical consequence: any stablecoin, settlement, or perps claim inherits a price-admission contract rather than merely trusting a floating external number.

This plane is wider than one oracle adapter. It includes:

- price provenance packets and optional attestation surfaces;
- active-vs-pending oracle transition discipline;
- quote freshness and settlement snapshot alignment;
- cross-module oracle alignment assumptions used by zUSD and perps.

The key architecture point is that oracle posture is not just a risk-layer detail. It constrains admission, settlement, and liquidation semantics across the system.

8 Runtime plane

The public runtime surface is deliberately narrower than the repository. The supported ingress contract is documented rather than inferred. Its public-safe reading is

$$\text{SupportedHTTP} \subset \text{AllRuntimeEndpoints}.$$

Standard reading: the supported public runtime path is a bounded subset of the possible API and integration surfaces. Practical consequence: a whole-system paper should describe the full runtime stack while still naming the narrower release-backed boundary.

The runtime plane itself includes:

- supported HTTP handlers for quote and settlement packet work;
- signer, nonce, and receipt binding surfaces;
- snapshot recovery and deterministic serialization boundaries;
- Tau testnet adapters and plugin shells;
- replay and gate wrappers used to rebuild the public assurance surface.

A representative runtime invariant is

$$\text{SignedOrWitnessedIngress} \rightarrow \text{NonceConsistent} \wedge \text{ReceiptConsistent} \wedge \text{CanonicalPayloadConsistent}.$$

Standard reading: admitted runtime ingress must bind signer, nonce, receipt, and canonical payload views consistently. Practical consequence: replay safety depends as much on ingress discipline as on execution math.

The replay-critical runtime subfamilies are:

1. request-envelope validation and intent authentication;
2. quote-receipt transport and stale-receipt rejection;
3. nonce state transitions, replay prevention, and sequence monotonicity;
4. snapshot encode/decode and restart recovery.

These are the surfaces where a public release can look “mathematically safe” on paper while still failing operationally if the ingress shells drift.

9 Risk and monetary planes

9.1 zUSD

The public zUSD posture is narrower than a full stablecoin completeness proof, but the intended invariant shape is already explicit:

$$\text{zUSDAdmissible} \rightarrow \text{DebtConservation} \wedge \text{CollateralConservation}.$$

The oracle safety side is shaped by

$$\text{price_pending} \neq \text{price} \rightarrow \text{RiskyTransitionsBlocked}.$$

Standard reading: zUSD actions are only admissible when both balance laws hold, and stale or pending oracle posture blocks risky transitions. Practical consequence: debt-only reasoning is not an adequate public safety story for the monetary layer.

The intended zUSD substructure is:

- mint, repay, withdraw-collateral, liquidation, and risky-ops gates;
- oracle-commit and multi-oracle alignment contracts;
- debt and collateral conservation helper theorems;
- runtime token and wallet boundaries that must preserve the same monetary semantics.

Standard reading: zUSD is not one theorem or one Python module. It is a family of balance laws, admission contracts, and oracle-state gates that must remain consistent across kernel, runtime, and wallet-adjacent surfaces. Practical consequence: a full-system paper has to describe the stablecoin plane as a composition problem, not only as a mint/redeem formula.

9.2 Perps

The intended perpetual-risk law is bounded rather than universal:

$$\text{PerpTransitionAllowed} \rightarrow \text{FreshOracleWindow} \wedge \text{BoundedMoveGate} \wedge \text{FundingOrSettlementWitnessOK}.$$

Standard reading: the perps layer relies on explicit oracle freshness, bounded moves, and witness-backed settlement or funding lanes. Practical consequence: the paper can cover the perps plane in detail without claiming that every derivatives surface is already release-authoritative.

The intended perps plane includes:

- funding epoch and settlement witness lanes;
- bounded-move and insurance-envelope guards;
- liquidation containment contracts;
- public API and settlement shells that mirror the same bounded semantics.

Standard reading: the perps plane is a bounded risk machine, not a generic unconstrained derivatives engine. Practical consequence: the correct public claim is a bounded envelope posture, not “all derivatives semantics are fully solved”.

10 Control and advisory surfaces

The full intended system contains non-authoritative control and preparation surfaces: policy compilation, operator configuration, safety review, and bounded recommendation layers. The public safety law is

$$\text{AdvisoryPlane} \rightarrow \text{NonAuthoritative}, \quad \neg(\text{AdvisoryPlane} \rightarrow \text{BalanceMovingAuthority}).$$

Standard reading: planning or ranking surfaces may influence what operators look at, but they do not gain direct value-moving authority. Practical consequence: the whole-system paper includes these layers because they shape the intended architecture, but they remain outside the core public RC1 execution claim.

11 Assurance plane

The assurance plane is the layer that turns the architecture into a replayable public claim. Its operating law is

$$\text{PublishedClaimOK} \rightarrow \text{ManifestPinned} \wedge \text{ReplaySurfaceTracked} \wedge \text{GeneratedEvidenceRebuildable}.$$

Standard reading: a publishable claim requires pinned manifests, tracked replay entry points, and the ability to rebuild local evidence from a clean checkout. Practical consequence: assurance is an executable surface, not just a set of markdown promises.

The major replay lanes are:

1. `python3tools/permissionless_assurance.pystatus`
2. `python3tools/permissionless_assurance.pyreplaypublic`
3. `python3tools/permissionless_assurance.pyreplaycritical`
4. `python3tools/permissionless_assurance.pyreplayfull`

They serve distinct purposes:

- *status* checks the publishable board and tracked lane inventory;
- *public* rebuilds the public proof surface;
- *critical* reruns the release-facing quality ratchets;
- *full* reruns the complete public release gate, including Tau, mutation, fuzz, and recovery checks.

Standard reading: the assurance plane itself is stratified. Practical consequence: the public branch can stay narrow without giving up whole-system replayability.

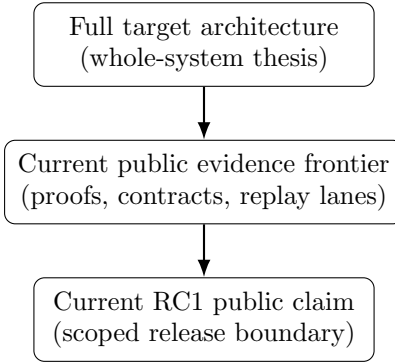


Figure 2: Assurance boundary picture. The target architecture is broader than the current public evidence frontier, and the RC1 public claim is narrower still.

12 Public assurance and RC1 boundary

The public assurance plane is defined by replayable manifests, release docs, claims registry entries, and gate commands rather than by prose alone. The public RC1 contract is

$$\text{RC1ClaimOK} := \text{CleanTree} \wedge \text{ScopeFrozen} \wedge \text{ReplayGreen} \wedge \text{ExclusionsHonest}.$$

Standard reading: RC1 is honest only when the tree is clean, the scope is explicit, the replay lanes are green, and excluded/disputed surfaces stay excluded. Practical consequence: a paper claim and a release claim are related, but not identical.

13 Guarantees and assumptions

14 Whole-system review checklist

For a reviewer trying to understand the full system rather than one isolated module, the practical order is:

1. review the execution and certificate planes together, because canonical winners only become settlement-authoritative through certificate shells;
2. review the oracle and risk planes together, because freshness and provenance posture are shared preconditions for zUSD, settlement, and perps;
3. review the runtime and control planes together, because nonce and ingress mistakes can invalidate otherwise correct core semantics;
4. review the assurance plane last, because it explains which of the above surfaces are presently release-backed versus only architectural targets.

Standard reading: the full system is best reviewed by dependency order rather than by directory order. Practical consequence: the paper is intentionally structured as a system map, not just as a list of files.

15 End-state proof agenda

The finished system still needs more than the current RC1 surface. The major remaining proof obligations are:

1. widen bounded canonicity theorems into broader generator and packet surfaces without losing explicit candidate-domain assumptions;
2. close more of settlement and price-provenance behavior inside kernel or theorem-backed packet shells;
3. strengthen zUSD and perps from mixed contract/replay posture into broader proof-backed public claims;
4. keep runtime ingress, signer, nonce, and replay surfaces aligned with the same canonical payload semantics proved or contracted below them;
5. preserve the distinction between the broad architecture target and the narrower release-backed public claim.

The public references that ground this paper are intentionally narrow:

- docs/PUBLIC_ASSURANCE_REPLAY.md
- docs/RC1_SCOPE.md

Surface	Intended guarantee	Current strongest public evidence	Main assumptions / limits
Spot winner selection	unique canonical winner on emitted bounded candidates	proved	explicit total key, nonempty admitted family, bounded emitted domain
Spot CPMM and fee core	reserve-bounded swap and exact accounting posture	proved	model-specific integer assumptions; dust state remains explicit
Settlement / quote binding	replay-verifiable canonical settlement packet	contract + proved helper shells + tests	certificate lanes must remain explicit and canonical
Value / price packets	meaningful value-neutral and provenance contracts	contract + tests	declared price and freshness inputs are trusted contract inputs
Kernel composition	inductive mirror-sync preservation	contract	cross-module facts must be synchronized, not assumed eventually consistent
Runtime ingress	signer / nonce / receipt consistency	contract + tests	supported HTTP boundary is narrower than the full runtime codebase
ZUSD / oracle safety	bounded solvency and stale-oracle fail-closed posture	mixed: proved helpers, contracts, tests	not every monetary claim is release-backed today
Perps / funding / liquidation	bounded risk-envelope posture	mixed: contracts, kernels, replay evidence	perps public claims remain narrower than full derivatives completeness
Public RC1 release	replayable clean-checkout assurance surface	contract + replay evidence	clean tree, frozen scope, green lanes, honest exclusions

Table 1: Whole-system guarantee matrix. The architecture is broad, but the public evidence class and assumption boundary vary by surface.

- docs/RC1_VERIFIED_SURFACE_MATRIX.md
- docs/TLA_CLAIM_SUMMARY.md
- docs/KERNEL_ABI_AND_COMPOSITION.md
- docs/claims_registry.yaml
- theorem-bearing files under lean-mathlib/Proofs/
- public kernels under src/kernels/dex/

That is the core honesty rule of this whitepaper. It describes the whole system in detail, but it only relies on public-safe artifacts and explicitly bounded public claims.